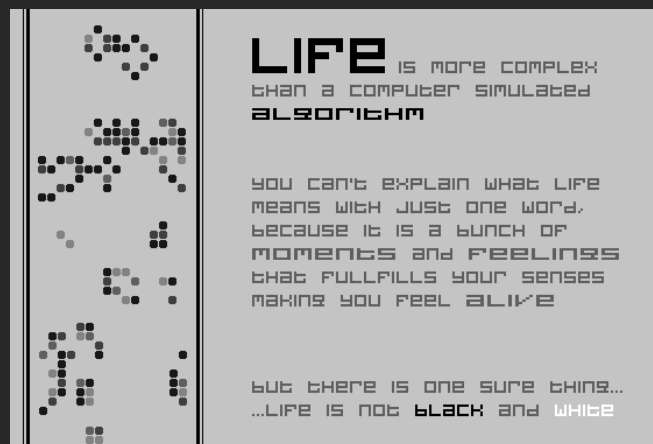


Inigo Quilez :: articles :: the game of life - 1998, 2016

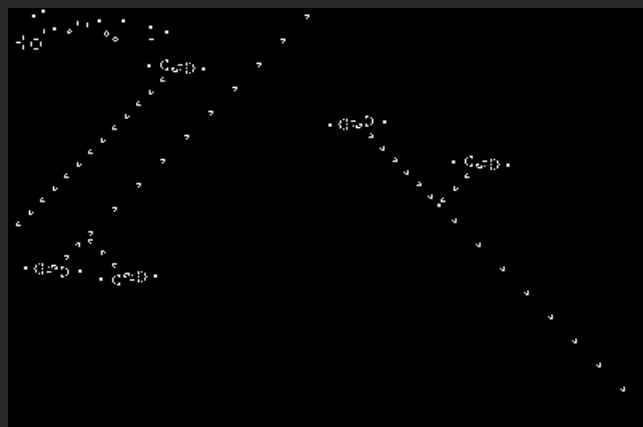


Conway's Game of Life is probably the best known Cellular Automata ever. Both Math and Computer Science profesional and aficionados have been deeply attracted by it because it's incredibly simple to describe and code (it's not even 10 lines), yet it produces amazingly complex patterns. In fact, it can reproduce a variety of structures than grow, move, die, and reproduce themselves, including functional structures such as clocks, logical gates, counters, etc, what allows at least to build computers within the universe of the game itslef.

You can probably read about Conway's Game of Life in many places, but in short, it goes like this: you have a 2D grid cells where each cell can be either dead (0) or alive (1). Then at each iteration of the algorithm (or "generation") the state of each cell will change depending on its current state and the state of the direct neighbors. There are two rules that drive this change: if a cell is alive and there is two or three surrounding cells are alive, then keep it alive, otherwise die. If the cell is already dead but three of the eight neighbour cells are alive, then the cell comes to life. Once all the cells of the grid have been updated this way, we have completed a new generation, and we can start over again. The process repeats for ever, and so the patterns of zeros and ones evolve forming the mesmerizing structurest. You need two buffers for this of course, one where you have the current generation of cells and another one where you are



"Life" 64 kilobytes demo, 2001.



Some clocks I made, using "glider gun" structures.

going to compute the new one, which you can keep ping-pong-ing to prevent memory copies or clears.

I wrote Game of Life back in the 90s like most nerd kids of the time because it is that easy. Later in 2001 my colleague Asier Murciego and I used the algorithm, as a metaphor for life in 2001 in this 64 kilobyte demo called "Life", which you can watch and download [here](#).

The core of the algorithm looks as simple as this:

```
// traditional
```

```
void updateCell( int i, int j )
{
    int k =  cell(i-1, j-1) + cell(i, j-1) + cell(i+1, j-1)
            + cell(i-1, j  ) +                cell(i+1, j  )
            + cell(i-1, j+1) + cell(i, j+1) + cell(i+1, j+1);

    int e = cell(i,j);
    if( (e==1) && (k==3 || k==2) ) return 1;
    if( (e==0) && (k==3      ) ) return 1;
    return 0;
}
```

Now, around 2016 I was thinking of the Game of Life again and how similar the neighborhood scan is to a regular 3x3 convolution. So I rewrote the algorithm in a couple of different ways, as a low pass filter and as a high pass filter followed by a fancy thresholding function:

```
// low pass filter
```

```
void updateCell( int i, int j )
{
    int k =  cell(px+ivec2(-1,-1)) + cell(px+ivec2(0,-1)) + cell(px+ivec2(1,-1))
            + cell(px+ivec2(-1, 0)) + cell(px)*9                + cell(px+ivec2(1, 0))
            + cell(px+ivec2(-1, 1)) + cell(px+ivec2(0, 1)) + cell(px+ivec2(1, 1));
    return (k==3 || k==11 || k==12) ? 1 : 0;
}
```

```
// high pass filter
void updateCell( int i, int j )
{
    int k =    cell(px+ivec2(-1,-1)) + cell(px+ivec2(0,-1)) + cell(px+ivec2(1,-1))
              + cell(px+ivec2(-1, 0)) - cell(px)*8              + cell(px+ivec2(1, 0))
              + cell(px+ivec2(-1, 1)) + cell(px+ivec2(0, 1)) + cell(px+ivec2(1, 1));
    return  (k== -6 || k== -5 || k== 3) ? 1 : 0;
}
```

You can also see this versions of the algorithm, as 3x3 convolution network followed by a funny activation function, if you really insist in seeing it all through the optics of machine learning that is.

You have a realtime running implementation with source code here in Shadertoy:

<https://www.shadertoy.com/view/XstGRf>.

inigo quilez - learning computer graphics since 1994